

Time Series Analysis & Visualization in Python

Project Overview

Time series analysis is the process of analyzing data collected over time to identify patterns, trends, and seasonal variations. It is widely used in industries such as finance, pharmaceuticals, social media, healthcare, and research to understand historical behavior and support future forecasting. By analyzing time-ordered data, organizations can make informed, data-driven decisions.

Dataset Summary

The dataset consists of observations recorded in chronological order at regular intervals. It contains time-based records that help analyze changes over time and identify recurring patterns, trends, and anomalies. The data may include either continuous values (e.g., stock prices or temperature) or discrete values (e.g., event counts or categorical statuses).

Dataset Information

- **Dataset Type:** Time Series Data
- **Format:** CSV
- **Nature of Data:** Chronologically ordered observations
- **Value Type:** Continuous and/or Discrete
- **Applications:** Forecasting, trend analysis, anomaly detection, and decision support

Objectives

- Analyze time-dependent data to identify meaningful patterns.
- Detect long-term trends and seasonal variations.
- Visualize temporal changes using suitable charts.
- Generate insights that support forecasting and decision-making.

Key Concepts

- **Trend:** Long-term increase, decrease, or stability in data.
 - **Seasonality:** Patterns that repeat at regular intervals.
 - **Moving Average:** Smooths short-term fluctuations to reveal trends.
 - **Noise:** Random variations without a consistent pattern.
 - **Differencing:** Calculates differences between consecutive observations to achieve stationarity.
 - **Stationarity:** A time series whose statistical properties remain constant over time.
 - **Order of Differencing:** Number of times differencing is applied to make the series stationary.
 - **Autocorrelation:** Measures the relationship between observations and their previous values.
 - **Resampling:** Changes the frequency of observations (daily to monthly, hourly to daily, etc.).
-

Types of Time Series Data

Continuous Time Series

Contains continuous numerical values recorded over time, such as stock prices, temperature, rainfall, or sensor readings.

Discrete Time Series

Contains countable or categorical values recorded at specific time intervals, such as daily customer visits, product sales, or website clicks.

Visualization Techniques

- **Line Chart:** Best for showing trends and fluctuations over time.
- **Area Chart:** Highlights cumulative changes over a period.
- **Bar Chart:** Useful for comparing discrete observations across time.
- **Histogram:** Displays the distribution of observations.

Time Series Visualization with Python :

Step 1: Installing and Importing Libraries

We will be using [Numpy](#), [Pandas](#), [seaborn](#) and [Matplotlib](#) libraries.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.tsa.stattools import adfuller
```

Step 2: Loading the Dataset

Here we will load the dataset and use the `parse_dates` parameter to convert the `Date` column to the `DatetimeIndex` format.

```
df = pd.read_csv("/content/stock_data.csv",
                 parse_dates=True,
                 index_col="Date")
df.head()
```

	Unnamed: 0	Open	High	Low	Close	Volume	Name
Date							
2006-01-03	NaN	39.69	41.22	38.79	40.91	24232729	AABA
2006-01-04	NaN	41.22	41.90	40.77	40.97	20553479	AABA
2006-01-05	NaN	40.93	41.73	40.85	41.53	12829610	AABA
2006-01-06	NaN	42.88	43.57	42.80	43.21	29422828	AABA
2006-01-09	NaN	43.10	43.66	42.82	43.42	16268338	AABA

Step 3: Cleaning of Data

We will drop columns from the dataset that are not important for our visualization.

```
df.drop(columns='Unnamed: 0', inplace=True)
df.head()
```

	Open	High	Low	Close	Volume	Name
Date						
2006-01-03	39.69	41.22	38.79	40.91	24232729	AABA
2006-01-04	41.22	41.90	40.77	40.97	20553479	AABA
2006-01-05	40.93	41.73	40.85	41.53	12829610	AABA
2006-01-06	42.88	43.57	42.80	43.21	29422828	AABA
2006-01-09	43.10	43.66	42.82	43.42	16268338	AABA

Step 4: Plotting High Stock Prices

Since the 'High' column is of continuous data type we will use line graph to visualize it.

- **sns.lineplot(data=df, x=df.index, y='High', label='High Price', color='blue')**: Plots High prices over time using the datetime index on x-axis.

```
sns.set(style="whitegrid")

plt.figure(figsize=(12, 6))
sns.lineplot(data=df, x='Date', y='High', label='High Price', color='blue')

plt.xlabel('Date')
plt.ylabel('High')
plt.title('Share Highest Price Over Time')

plt.show()
```



Step 5: Resampling Data

To better understand the trend of the data we will use the resampling method which provide a clearer view of trends and patterns when we are dealing with daily data.

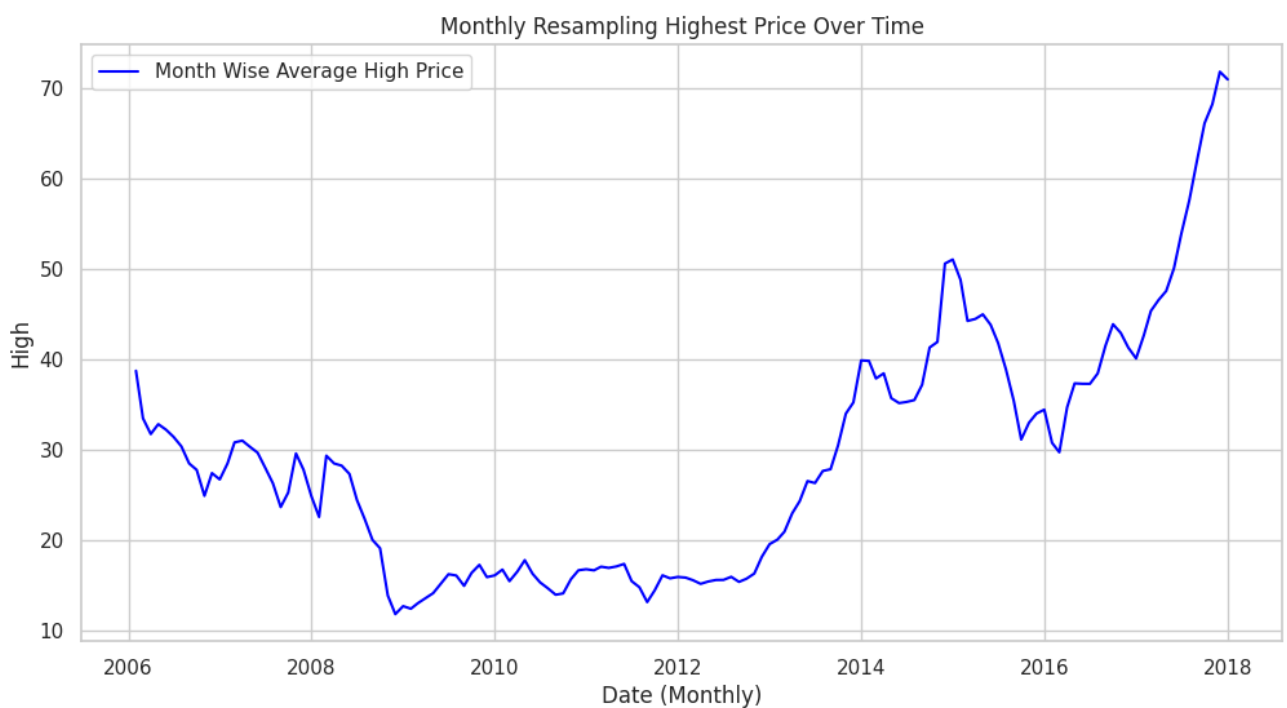
- **df_resampled = df.resample('ME').mean(numeric_only=True):** Resamples data to monthly frequency and calculates the mean of all numeric columns for each month.

```
df_resampled = df.resample('M').mean(numeric_only=True)

sns.set(style="whitegrid")

plt.figure(figsize=(12, 6))
sns.lineplot(data=df_resampled, x=df_resampled.index, y='High', label='Month Wise Average High Price', color='blue')

plt.xlabel('Date (Monthly)')
plt.ylabel('High')
plt.title('Monthly Resampling Highest Price Over Time')
plt.show()
```



Step 6: Detecting Seasonality with Autocorrelation

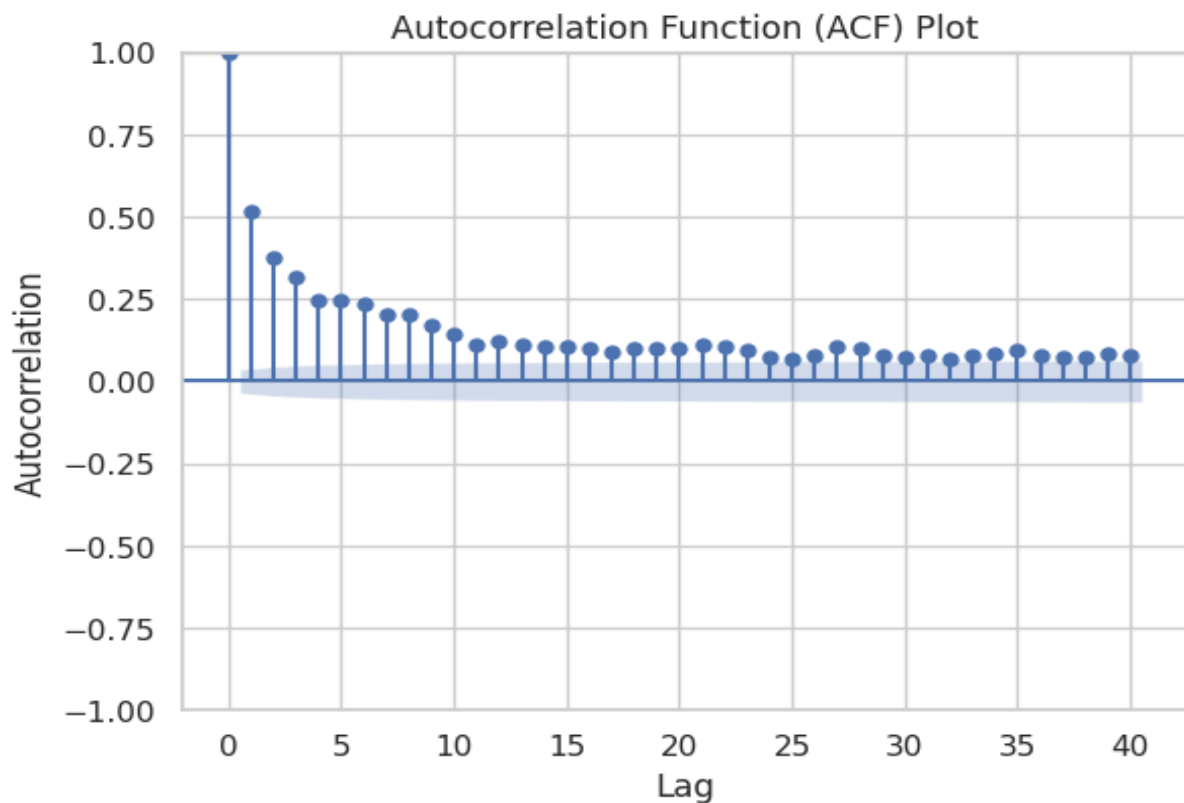
We will detect Seasonality using the autocorrelation function (ACF) plot. Peaks at regular intervals in the ACF plot suggest the presence of seasonality.

```

if 'Date' not in df.columns:
    print("'Date' is already the index or not present in the DataFrame.")
else:
    df.set_index('Date', inplace=True)

plt.figure(figsize=(12, 6))
plot_acf(df['Volume'], lags=40)
plt.xlabel('Lag')
plt.ylabel('Autocorrelation')
plt.title('Autocorrelation Function (ACF) Plot')
plt.show()

```



Step 7: Testing Stationarity with ADF test

We will perform the [ADF test](#) to formally test for stationarity.

```

from statsmodels.tsa.stattools import adfuller

result = adfuller(df['High'])
print('ADF Statistic:', result[0])
print('p-value:', result[1])
print('Critical Values:', result[4])

```

```

ADF Statistic: 0.7671404880535936
p-value: 0.9910868050318213
Critical Values: {'1%': np.float64(-3.4325316347197403), '5%': np.float64(-2.862503905260741), '10%': np.float64(-2.5672831121111113)}

```

- Based on the ADF Statistic we accept the null hypothesis, indicating that the data is not stationary according to the Augmented Dickey-Fuller test.
- This suggests that differencing or other transformations may be needed to achieve stationarity before applying certain time series models.

Step 8: Differencing to Achieve Stationarity

Differencing involves subtracting the previous observation from the current observation to remove trends or seasonality.

```
df['high_diff'] = df['High'].diff()

plt.figure(figsize=(12, 6))
plt.plot(df['High'], label='Original High', color='blue')
plt.plot(df['high_diff'], label='Differenced High', linestyle='--', color='green')
plt.legend()
plt.title('Original vs Differenced High')
plt.show()
```



Step 9: Smoothing Data with Moving Average

`df['High'].diff()`: helps in calculating the difference between consecutive values in the High column. This differencing operation is used to transform a time series into a new series that represents the changes between consecutive observations.

```

window_size = 120
df['high_smoothed'] = df['High'].rolling(window=window_size).mean()

plt.figure(figsize=(12, 6))

plt.plot(df['High'], label='Original High', color='blue')
plt.plot(df['high_smoothed'], label=f'Moving Average (Window={window_size})', linestyle='--', color='orange')

plt.xlabel('Date')
plt.ylabel('High')
plt.title('Original vs Moving Average')
plt.legend()
plt.show()

```



This calculates the moving average of the High column with a window size of 120(A quarter), creating a smoother curve in the **high_smoothed** series. The plot compares the original High values with the smoothed version.

Step 10: Original Data vs. Differenced Data

Printing the original and differenced data side by side we get:

```

df_combined = pd.concat([df['High'], df['high_diff']], axis=1)
print(df_combined.head())

```

	High	high_diff
Date		
2006-01-03	41.22	NaN
2006-01-04	41.90	0.68
2006-01-05	41.73	-0.17
2006-01-06	43.57	1.84
2006-01-09	43.66	0.09

Hence the `high_diff` column represents the differences between consecutive high values. The first value of `high_diff` is NaN because there is no previous value to calculate the difference.

As there is a NaN value we will drop that and proceed with our test:

```
df.dropna(subset=['high_diff'], inplace=True)
df['high_diff'].head()
```

	high_diff
Date	
2006-01-04	0.68
2006-01-05	-0.17
2006-01-06	1.84
2006-01-09	0.09
2006-01-10	-0.32

dtype: float64

After that if we conduct the ADF test:

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(df['high_diff'])
print('ADF Statistic:', result[0])
print('p-value:', result[1])
print('Critical Values:', result[4])
```

ADF Statistic: -12.14836747834324

p-value: 1.5912766134149499e-22

Critical Values: {'1%': np.float64(-3.4325316347197403), '5%': np.float64(-2.862503905260741), '10%': np.float64(-2.5672831121111113)}

Since the p-value is less than 0.05, we reject the null hypothesis and conclude that the series is stationary.